

Lindström Quantifiers and First-Order Complexity

by
Quinlan Mayo

A thesis submitted in partial fulfillment
of the requirements for the degree of
Bachelor of Science
(Computer Science & Mathematics)

at the
University of Massachusetts Amherst
2025

The thesis is approved by the following members of the Thesis Committee: Eric Allender, David A. Mix Barrington.

Abstract

We study the use of generalized quantifiers (Lindström quantifiers) within descriptive complexity in an effort to give a more unified presentation of first-order logics corresponding to circuit classes. Within the same framework, we also examine the hardness of regular and visibly counter languages within $\mathbf{NC}^1$.

Acknowledgements

We thank Eric Allender and Dave Barrington for helpful technical advice and assistance in improving the presentation of some material.

Contents

1	Introduction	2
1.1	Overview	2
1.1.1	Background	2
1.1.2	Summary of Results	3
1.2	Preliminaries	3
1.2.1	Algebra	3
1.2.2	Descriptive Complexity	4
1.2.3	Visibly Counter Automata	5
1.2.4	Complexity Classes	6
2	Lindström Quantifiers and Circuit Complexity	8
2.1	Lindström Quantifiers	8
2.1.1	Generalized Quantifiers	8
2.1.2	Extensions of FO & Circuit Uniformity	11
2.2	Monoidal Quantifiers and NC ¹	12
2.2.1	Regular Lindström Quantifiers and Finite Monoids.	12
2.2.2	Logical Characterizations of NC ¹ its Subclasses	16
2.2.3	Monoidal Extensions of FO without BIT	18
2.3	Groupoidal Quantifiers and LOGCFL	19
2.3.1	Logical Characterizations of LOGCFL	19
2.3.2	Unary Groupoidal Extensions of FO without BIT	21
2.3.3	Unary Groupoidal Extensions of FO with BIT	23
2.4	The First-Order Landscape of LOGCFL and Open Problems	24
3	The Complexity of Some Formal Language Families	25
3.1	Regular Languages	25
3.2	Visibly Counter Languages	29
3.2.1	Height Computation	30
3.2.2	Regular Part Computation	32
3.2.3	Hardness Results	33
4	Conclusion	35

Chapter 1

Introduction

1.1 Overview

1.1.1 Background

The connection between circuits and automata has been a topic of much interest in small-depth circuit complexity. The seminal works of Barrington and Theri  n established a connection between $\mathbf{NC}^1$ circuits, bounded-width branching programs, and finite automata [2, 3]. Barrington, Immerman, and Straubing [14] then provided an elegant characterization of $\mathbf{NC}^1$ in terms of first-order logics augmented by generalized quantifiers over monoids. Straubing [17] later gave a comprehensive account of this theory, identifying its central relevance in understanding $\mathbf{NC}^1$. Lautemann, McKenzie, Schwentick, and Vollmer [13], proceeding along similar lines, developed a descriptive theory of **LOGCFL**. Unlike previous works, they explicitly utilized the generalized quantifiers of Lindstr  m [7], which resulted in a more fine-grained analysis of augmented first-order logics; questions of the computational power of quantifier arity and nestedness were considered.

In a related direction, Barrington, Compton, Straubing, and Theri  n [4] investigated reducibility relations among regular languages with an application to the regular languages in certain circuit classes. They showed that the monoids appearing in the image of the syntactic morphism of a regular language can be “simulated” by that language, establishing

completeness results for $\mathbf{NC}^1$ and some of its subclasses. Following in their footsteps, Krebs, Lange, Ludwig, and Hahn [12, 9] generalized these results to the class visibly counter languages, identifying complete languages for even more subclasses of $\mathbf{NC}^1$. In this thesis, we survey some important results in these two strands of research.

1.1.2 Summary of Results

In Chapter 2, we review the main results of the aforementioned works, formulating them as statements about first-order logics supplemented with Lindström quantifiers. We show that the monoidal quantifiers of Barrington *et al.* in [14] are equivalent to Lindström quantifiers over regular languages. We also note that their main theorem on circuit uniformity still holds in the more general setting of first-order logic with Lindström quantifiers, with minor restrictions. Finally, we identify some open problems in the descriptive theory of $\mathbf{NC}^1$ and **LOGCFL**.

In Chapter 3, we reproduce the proofs of the main results of [4] on the complexity of regular languages, borrowing some notation from [9] to improve readability. We also state the results of [12, 9] on the complexity of visibly counter languages, which identify a new family of formal languages that are $\mathbf{TC}^0$ -complete.

1.2 Preliminaries

1.2.1 Algebra

Let Σ be a finite alphabet and $L \subseteq \Sigma^*$. The *syntactic congruence* of L is a relation $\sim_L$ on Σ^* defined by $x \sim_L y$ iff for all $u, v \in \Sigma^*$, we have $uxv \in L \iff uyv \in L$. The *syntactic monoid* of L is the quotient monoid $\Sigma^* / \sim_L$. The *syntactic morphism* of L is the projection morphism $\eta_L : \Sigma^* \rightarrow \Sigma^* / \sim_L$.

Let G be a group. Then the subgroup $G^{(1)}$ of G is the commutator subgroup of G , i.e. elements of the form $ghg^{-1}h^{-1}$ for $g, h \in G$. We define $G^{(n)}$ to be $(G^{(n-1)})^{(1)}$. The group G is called *solvable* if there exists some $n > 0$ such that $G^{(n)} = \{1_G\}$. A monoid is

called solvable if all of its subgroups are solvable. If a group or monoid is not solvable, it is called *unsolvable*.

Let M, N be monoids. We say M *divides* N iff there is a subset $N' \subseteq N$ admitting a surjective monoid homomorphism $N' \rightarrow M$. A monoid is said to be *simple* if it has no nontrivial monoid congruences. The simple monoids are the simple groups and the commutative aperiodic unit $U_1 = (\{0, 1\}^*, \cdot)$ where $u \cdot 0 = 0$ and $u \cdot 1 = 1$ for all $u \in U_1$. If M is a finite monoid, a *word problem* over M , denoted $\mathcal{WP}(M, T)$ for some $T \subseteq M$, is the set of all words in M^* which multiply to an element of T . When $T = \{1_M\}$, we may simply write $\mathcal{WP}(M)$, which we sometimes call *the* word problem over M .

1.2.2 Descriptive Complexity

A *vocabulary* $\tau = \langle R_1^{a_1}, \dots, R_r^{a_r}, c_1, \dots, c_s \rangle$ is a tuple of relation symbols R_i with arity a_i and constant symbols c_i . A *structure* $\mathcal{S} = \langle \{0, 1, \dots, n-1\}, R_1, \dots, R_r, c_1, \dots, c_s \rangle$ of type τ is a tuple containing a universe $|\mathcal{S}| = \{0, 1, \dots, n-1\}$, relations R_i which take a_i arguments from $|\mathcal{S}|$, and constant symbols $c_i \in |\mathcal{S}|$. Let $\text{STRUC}[\tau]$ denote the set of all finite structures of type τ , and let $\mathcal{L}(\tau)$ denote the set of logical formulas formed with symbols from τ , binary numerical predicates and constants $=, \leq, \text{BIT}, 0, \mathbf{m}$, logical connectives $\wedge, \vee, \neg$, variables $x_1, x_2, \dots$, and first-order quantifiers $\forall$ and $\exists$. A subset of $\text{STRUC}[\tau]$ is called a *problem*.

We will deal primarily with structures encoding words over an alphabet $\Sigma = \{a_1, \dots, a_s\}$, which have type $\tau = \langle R_1^1, \dots, R_{s-1}^1 \rangle$. The “ i -th letter of structure $\mathcal{S} \in \text{STRUC}[\tau]$ ” is usually understood to be a_j if $\mathcal{S} \models R_j(i)$. However we find it convenient to adopt the interpretation that the i -th letter of $\mathcal{S}$ is a_j if j is the smallest integer such that $\mathcal{S} \models R_j(i)$, or a_s if no such j exists.

Let σ and τ be vocabularies where $\tau = \langle R_1^{a_1}, \dots, R_r^{a_r} \rangle^1$ and let $S \subseteq \text{STRUC}[\sigma]$ and $T \subseteq \text{STRUC}[\tau]$ be problems. We say that S is *first-order reducible* to T , denoted $S \leq_{\text{FO}} T$, iff there exists a mapping $I : \text{STRUC}[\sigma] \rightarrow \text{STRUC}[\tau]$ such that

¹We could easily extend this definition to deal with vocabularies containing constants but we do not need that in this paper.

1. There exist an integer $k > 0$ (the arity of the reduction) and formulas $\varphi_{R_1}, \dots, \varphi_{R_r} \in \mathcal{L}(\sigma)$ where φ_{R_i} has arity $k \cdot a_i$ such that for every $\mathcal{S} \in \text{STRUC}[\sigma]$, the structure $I(\mathcal{S}) \in \text{STRUC}[\tau]$ is defined as follows:
 - The universe of $I(\mathcal{S})$ is $|\mathcal{S}|^k$.
 - For each relation symbol R_i in the vocabulary τ , the interpretation of R_i as a subset of $|I(\mathcal{S})|^{a_i}$ is given by $\{(\overline{s_1}, \dots, \overline{s_{a_i}}) \in |I(\mathcal{S})|^{a_i} \mid \mathcal{S} \models \varphi_{R_i}(\overline{s_1}, \dots, \overline{s_{a_i}})\}$.
2. For all $\mathcal{S} \in \text{STRUC}[\sigma]$, we have $\mathcal{S} \in S \iff I(\mathcal{S}) \in T$.

Furthermore, we write $I = \lambda\langle\varphi_1, \dots, \varphi_r\rangle$ and call I a *first-order reduction*.

1.2.3 Visibly Counter Automata

Bárány, Löding, and Serre [1] introduced *visibly counter automata*. The input alphabet $\hat{\Sigma} = \Sigma_{\text{call}} \sqcup \Sigma_{\text{ret}} \sqcup \Sigma_{\text{int}}$ is partitioned into call, return, and internal letters. We refer to such an alphabet as a *visibly pushdown alphabet*. When a call/return/internal letter is read, the automaton's counter must increment/decrement/idle, respectively. The height of a word $w \in \hat{\Sigma}^*$, denoted $\Delta(w)$, is given by the number of call letters in w minus the number of return letters. A word is *well-matched* if every prefix has non-negative height.

Definition 1.2.1. An m -VCA $\mathcal{A}$ over a visibly pushdown alphabet $\hat{\Sigma}$ is a tuple $\mathcal{A} = (Q, q_0, F, \hat{\Sigma}, \delta_0, \dots, \delta_m)$, where $m \geq 0$ is the threshold, Q is the set of states, $q_0 \in Q$ is the start state, F is the set of final states, and $\delta_i : Q \times \Sigma \rightarrow Q$ are the transition functions. $\blacktriangle$

A *configuration* of an m -VCA is a state and a counter value, i.e. an element of $Q \times \mathbb{N}$. The initial configuration is $(q_0, 0)$. When reading a letter $a \in \hat{\Sigma}$ from configuration (q, h) , the VCA transitions to the configuration $(\delta_{\min(m, h)}(q, a), h + \Delta(a))$. We write $(q, h) \xrightarrow{w} (q', h')$ if $w \in \hat{\Sigma}^*$ takes the VCA from configuration (q, h) to (q', h') . The VCA accepts its input w if $(q_0, 0) \xrightarrow{w} (f, h)$ for some $f \in F$ and $h \geq 0$.

Definition 1.2.2 (VCL). The class of visibly counter languages, denoted **VCL**, is the set of languages recognized by an m -VCA for some m . $\blacktriangle$

1.2.4 Complexity Classes

We consider complexity classes defined by Boolean circuits, automata, and first-order formulas. In particular,

- **REG** is the set of regular languages.
- **VCL** is the set of visibly counter languages. (See Section 1.2.3)
- **CFL** is the set of context-free languages.
- **AC**⁰ is the set of languages with circuit families of polynomial size, constant depth, and unbounded fan-in Boolean gates.
- **FO** is the set of languages expressible by a first-order sentence.
- **ACC**⁰[$m_1, \dots, m_k$] is the set of languages with **AC**⁰ circuit families permitting modulo- m_i gates ($1 \leq i \leq k$). A modulo- m_i gate returns 1 iff the number of “1” inputs divisible by m_i . **ACC**⁰ is the union of all **ACC**⁰[m].
- **TC**⁰ is the set of languages with **AC**⁰ circuit families permitting majority gates. A majority gate returns 1 iff the number of “1” inputs is strictly greater than the number of “0” inputs.
- **NC**⁰ is the set of languages with circuit families of constant depth and fan-in 2 Boolean gates.
- **NC**¹ is the set of languages with circuit families of logarithmic depth and fan-in 2 Boolean gates.
- **SAC**¹ is the set of languages with circuit families of polynomial size, logarithmic depth, fan-in 2 $\wedge$ gates, and unbounded fan-in $\vee$ gates. Negated inputs are also supplied, but $\neg$ are prohibited elsewhere.
- **LOGCFL** is the set of languages logspace-reducible to a context-free language. It is equal to logspace-uniform **SAC**¹ [18].

The circuit classes here are defined non-uniformly. In particular, certain undecidable languages, such as undecidable unary languages, lie in these classes. We can impose uniformity conditions by insisting that a description of the n -th circuit in a circuit family can be produced by a (resource-constrained) Turing machine on input 1^n . In this paper, we consider restrictions of these classes by logspace uniformity and the logtime uniformity of [14].

A circuit class $\mathcal{C}$ can be augmented with oracles for a binary language L ; we denote this by $\mathcal{C}[L]$. Similarly, the class **FO** can be augmented with numerical predicates $N_1, \dots, N_k$: **FO** $[N_1, \dots, N_k]$ is the set of languages expressible by a first-order sentence permitting the use of predicates $N_1, \dots, N_k$. We use **ARB** to denote the set of all numerical predicates.

Chapter 2

Lindström Quantifiers and Circuit Complexity

2.1 Lindström Quantifiers

In the combinatorial setting, we can characterize "nice" circuit complexity classes¹ as $\mathbf{AC}^0[\mathcal{F}]$ for an appropriate family of boolean functions $\mathcal{F}$. For instance, $\mathbf{TC}^0$ is the $\mathbf{AC}^0$ -closure of $\text{MAJ} = \{\text{MAJ}_n\}_{n \in \mathbb{N}}$, and $\mathbf{NC}^1$ is the $\mathbf{AC}^0$ -closure of a family of functions computing a word problem of the permutation group S_5 [14]. In the descriptive approach, we might expect a succinct characterization of $\mathbf{AC}^0[\mathcal{F}]$ in terms of first-order logic since $\mathbf{AC}^0 = \mathbf{FO}[\text{ARB}]$ (c.f., e.g., [10]). Indeed, this is achieved by permitting generalized quantifiers, introduced by Lindström in [7], and used fruitfully by Straubing in [17]. In this section we define *Lindström quantifiers*, which provide a convenient way to express many complexity classes and facts about the structure of these classes.

2.1.1 Generalized Quantifiers

For the entirety of this section, all structures are assumed to be finite and of a fixed vocabulary σ , unless otherwise specified. Let $\varphi \in \mathcal{L}(\sigma)$ be a first-order formula with k free

¹A "nice" class is one with a complete problem under $\mathbf{AC}^0$ Turing reductions.

variables. Then for every structure $\mathcal{A}$, the formula φ induces a set $\varphi^{\mathcal{A}} = \{\bar{a} \in |\mathcal{A}|^k \mid \varphi(\bar{a})\}$, i.e. the set of all assignments satisfying φ in $\mathcal{A}$. Now let ψ be a formula with a single free variable x . The truth values of the formulae $\forall x : \psi(x)$ and $\exists x : \psi(x)$ are typically defined in the following way:

1. $\mathcal{A} \models \forall x : \psi(x) \iff \psi(a)$ is true for all $a \in |\mathcal{A}|$.
2. $\mathcal{A} \models \exists x : \psi(x) \iff \psi(a)$ is true for at least one $a \in |\mathcal{A}|$.

Equivalently, we may say $\psi^{\mathcal{A}} = |\mathcal{A}|$ instead of “ $\psi(a)$ is true for all $a \in |\mathcal{A}|$ ”, and $\psi^{\mathcal{A}} \neq \emptyset$ instead of “ $\psi(a)$ is true for at least one $a \in |\mathcal{A}|$ ”. Naturally, we can generalize first-order quantifiers by imposing arbitrary conditions on $\psi^{\mathcal{A}}$. We can further allow quantifiers to bind multiple variables, and operate on tuples of formulae. Syntactically, a k -ary quantifier Q acting on s formulae (of no particular arity) looks like

$$Qx_1, \dots, x_k : \langle \varphi_1(x_1, \dots, x_{i_1}), \varphi_2(x_{i_1+1}, \dots, x_{i_2}), \dots, \varphi_s(x_{i_{s-1}+1}, \dots, x_k) \rangle. \quad (2.1)$$

Semantically, a structure $\mathcal{A}$ satisfies Equation (2.1) iff some set of conditions on $\varphi_1^{\mathcal{A}}, \dots, \varphi_s^{\mathcal{A}}$ are met.

Just as we introduce oracles for Turing machines and circuits, we would like to augment some base logic (such as **FO**) to express properties of structures that would otherwise not be expressible in that logic. Generalized quantifiers would seem to be an appropriate tool; one can, for instance, strengthen **FO** by introducing a modular counting quantifier $\exists^{0,2}$ which can express the property that a binary string has an even number of 1’s [4, 17]. This property is not first-order, as $\text{PARITY} \notin \mathbf{AC}^0 = \mathbf{FO}[\text{ARB}]$ [8]. The following quantifier definition, due essentially to Lindström [7], allows us to extend the first-order system to express an arbitrary class of structures.

Definition 2.1.1 (First-Order Lindström Quantifiers). Let $\sigma = \langle <, R_1^{a_1}, R_2^{a_2}, \dots, R_s^{a_s}, c_1, \dots, c_t \rangle$ be a vocabulary and let $\mathcal{C} \subseteq \text{STRUC}[\sigma]$ be a class of σ -structures. Furthermore, let $\varphi_1, \dots, \varphi_s$ be formulas over a vocabulary τ (with ordering) such that φ_i has a_i free vari-

ables. Then the expression

$$Q_{\mathcal{C}}\overline{x_1}, \dots, \overline{x_s} : \langle \varphi_1(\overline{x_1}), \dots, \varphi_s(\overline{x_s}) \rangle \quad (2.2)$$

where each x_i is an a_i -tuple is a valid formula. For all τ -structures $\mathcal{A}$, Equation (2.2) is satisfied by $\mathcal{A}$ iff $\langle |\mathcal{A}|, <^{\mathcal{A}}, \varphi_1^{\mathcal{A}}, \dots, \varphi_s^{\mathcal{A}} \rangle \in \mathcal{C}$. $\blacktriangle$

As a (not particularly interesting) example, letting $\mathcal{C}$ be as in Definition 2.1.1, it is easy to see that $Q_{\mathcal{C}}\overline{x_1}, \dots, \overline{x_s} : \langle R_1(\overline{x_1}), \dots, R_s(\overline{x_s}) \rangle$ is satisfied by a structure $\mathcal{A} \in \text{STRUC}[\sigma]$ iff $\mathcal{A} \in \mathcal{C}$.

Since in complexity theory we typically deal with languages over a finite alphabet, it will be convenient to define *Lindström quantifiers over languages*. Such quantifiers are equivalent in power to normal Lindström quantifiers,² but they offer an appropriate formalism to easily translate between descriptive and combinatorial complexity. For simplicity, we identify a word over a finite alphabet with its corresponding finite model, so we may speak of strings satisfying logical formulas.

Definition 2.1.2 (Lindström Quantifiers Over a Language). Let $\Sigma = \{a_1, \dots, a_s\}$ be a finite alphabet with a corresponding vocabulary σ , and let $L \subseteq \Sigma^*$ be a language. Let $\varphi_1, \dots, \varphi_{s-1} \in \mathcal{L}(\sigma)$ be formulas with k free variables $\overline{x}$. Then the expression

$$Q_L \overline{x} : \langle \varphi_1(\overline{x}), \dots, \varphi_{s-1}(\overline{x}) \rangle \quad (2.3)$$

is a valid formula. A word $w \in \Sigma^n$ satisfies (2.3) iff $\lambda \langle \varphi_1, \dots, \varphi_{s-1} \rangle(w) \in L$. $\blacktriangle$

As before, it is easy to see that extending **FO** by Q_L immediately allows us to express the membership of a word in L . If the reader finds the motivation or finer details of Definitions 2.1.1 and 2.1.2 to be unclear, the following sections will contain concrete examples which more clearly elucidate the usefulness of these quantifiers.

²Evidently they are just as powerful as Lindström quantifiers over word structures, and every finite structure can be interpreted as a word structure by a first-order reduction [10].

2.1.2 Extensions of FO & Circuit Uniformity

Let L be a language and Q_L be the Lindström quantifier over L . We define $Q_L(\mathbf{FO})$ as the class of languages expressible by applying Q_L to an appropriate tuple of first-order formulae. If Q_L is a unary quantifier, we write $Q_L^*(\mathbf{FO})$. If $\mathcal{Q}$ is a *set* of Lindström quantifiers, then $\mathcal{Q}(\mathbf{FO})$ is the union of $Q_L(\mathbf{FO})$ for all $Q_L \in \mathcal{Q}$. We also define $\mathbf{FO}(Q_L)$ to be the class of languages expressible by first-order formulas making arbitrary use of Q_L (i.e., just as they typically make use of $\forall, \exists$). The classes $\mathbf{FO}(Q_L^*)$ and $\mathbf{FO}(\mathcal{Q})$ likewise denote $\mathbf{FO}(Q_L)$ when Q_L is unary, and the union of $\mathbf{FO}(Q_L)$ for $Q_L \in \mathcal{Q}$, respectively.

We remark that a complexity class $\mathcal{C}$ that is closed under $\leq_{fo}$ reductions is captured by $Q_L(\mathbf{FO})$ for some language L if and only if L is $\mathcal{C}$ -complete under first-order reductions. Naturally, we would like to make a similar statement when $\mathcal{C} = \mathbf{FO}(Q_L)$. Following Immerman and Landau [11], we define *first-order Turing reductions*:

Definition 2.1.3 (First-Order Turing Reductions). For any two languages L_1 and L_2 , we say L_1 is *first-order Turing-reducible* to L_2 if $L_1 \in \mathbf{FO}(Q_{L_2})$. In this case we write $L_1 \leq_{\mathbf{FO}}^t L_2$. ▲

Hence a complexity class $\mathcal{C}$ that is closed under $\leq_{\mathbf{FO}}^t$ reductions is captured by $\mathbf{FO}(Q_L)$ if and only if L is $\mathcal{C}$ -complete under first-order Turing reductions.

It was shown by Barrington, Immerman, and Straubing [14] that for a set $\mathcal{Q} = \{Q_{f_1}, \dots, Q_{f_n}\}$ of generalized quantifiers defined by certain *monoidal* boolean functions $f_1, \dots, f_n$, the logic $\mathbf{FO}(\mathcal{Q})$ coincides with **DLOGTIME**-uniform $\mathbf{AC}^0$ with oracle gates for $f_1, \dots, f_n$. Lautemann, McKenzie, Schwentick, and Vollmer observed in [13] that such quantifiers are in fact just special types of Lindström quantifiers. This result of Barrington *et al.* naturally generalizes to a much larger set of Lindström quantifiers as the following theorem states.

Theorem 2.1.1 ([14], Generalized Circuit Uniformity Theorem). *Let $L_1, \dots, L_n \subseteq \Sigma^*$ be languages with a neutral letter, let $\mathcal{F} = \{f_1, \dots, f_n\}$ be the set of characteristic functions of these languages so that $L_i = \{w \in \Sigma^* \mid f_i(w) = 1\}$, and let $\mathcal{Q} = \{Q_{L_1}, \dots, Q_{L_n}\}$ be the set of Lindström quantifiers over $L_1, \dots, L_n$. Then $\mathbf{FO}(\mathcal{Q}) = \mathbf{DLOGTIME}$ -uniform $\mathbf{AC}^0[\mathcal{F}]$.*

Proof. The proof of Theorem 8.1 (Main Theorem) in [14] carries through without issue. $\square$

The technical requirement that the languages we quantify over have a neutral letter is mainly an artifact of the way in which first-order reductions are defined. Such reductions are highly restrictive in the sense that a k -ary reduction maps a word of length n to a word of *exactly* length n^k . Without a neutral letter, it is thus unclear how the construction of Barrington *et al.* witnessing a $\mathbf{FO}(\mathcal{Q})$ formula for a uniform $\mathbf{AC}^0[\mathcal{F}]$ circuit family can be carried out. In practice, we are often quantifying over some kind of algebraic structure which has a unity element acting as the neutral letter; else we may simply add a padding character to the language so that the restriction is not an obstacle.

With Theorem 2.1.1 in mind, we remark that our notions of first-order reducibility ($\leq_{\mathbf{FO}}$ and $\leq_{\mathbf{FO}}^t$) thus corresponds to uniform- $\mathbf{AC}^0$ reducibility in the right circumstances. That is, for languages $A, B \subseteq \Sigma^*$ with a neutral letter, we have $A \leq_{\mathbf{FO}} B$ ($A \leq_{\mathbf{FO}}^t B$) if and only if $A \leq_{u\mathbf{AC}^0}^m B$ ($A \leq_{u\mathbf{AC}^0}^t B$) where $u\mathbf{AC}^0$ refers to $\mathbf{DLOGTIME}$ -uniform $\mathbf{AC}^0$.

2.2 Monoidal Quantifiers and $\mathbf{NC}^1$

Barrington *et al.* in [14] introduced *monoidal functions*, and showed that *monoidal quantifiers* defined by these functions yield elegant characterizations of $\mathbf{ACC}^0$, $\mathbf{TC}^0$, and $\mathbf{NC}^1$. In this section we review their definitions and summarize their key results within our framework.

2.2.1 Regular Lindström Quantifiers and Finite Monoids.

Straubing, Therien, and Thomas introduced *modular counting quantifiers* in [16], which count the number of satisfying assignments to a formula modulo some constant. Barrington *et al.* set out to extend such quantifiers to sets with an associative operation (monoids) in general. The following definition captures when a family of boolean functions is determined by the multiplication table of a monoid.

Definition 2.2.1 (Monoidal Functions [14]). Let k, ℓ be fixed integers and let $f = \{f_1, f_2, \dots\}$ be a family of functions where f_n is from $\{0, 1\}^{\ell \cdot n^k}$ to $\{0, 1\}$. We say that f is a k -ary monoidal function if there exists a monoid M and a subset T of M containing at least $1_M \in M$ such that for some map $m : \{0, 1\}^\ell \rightarrow M$ and all n , we have

$$f_n(\langle b_{1,1}, \dots, b_{1,\ell} \rangle, \dots, \langle b_{n^k,1}, \dots, b_{n^k,\ell} \rangle) = 1 \iff \prod_{i=1}^{n^k} m(b_{i,1}, \dots, b_{i,\ell}) \in T \quad (2.4)$$

That is, $f(\overline{b_1}, \dots, \overline{b_{n^k}}) = 1$ iff the images of the inputs under m multiply out (in order of inputs) to some element of T . ▲

Although we technically allow the monoid M in Definition 2.2.1 to be infinite, we will almost always be concerned with monoidal functions over *finite monoids*, which are well-studied objects (our analysis of the class $\mathbf{TC}^0$ is the lone exception in this paper).

Given a family of monoidal functions f , we define a quantifier Q_f such that Q_f applied to an appropriate ℓ -tuple of formulas is satisfied by a word w if and only if the successive signatures of the ℓ -tuple when evaluated in w on elements of the universe multiply out to a “good” monoid element when m is applied.

Definition 2.2.2 (Monoidal Quantifiers [14]). Let $f = \{f_1, f_2, \dots\}$ be a monoidal function and let $\varphi_1, \dots, \varphi_\ell$ be formulae that share k free variables $\bar{x} = \langle x_1, \dots, x_k \rangle$. Let w be a word of length n and let $P_1, P_2, \dots, P_{n^k}$ be all the elements of $\{1, \dots, n\}^k$ in lexicographic order. Then the expression

$$Q_f \bar{x} : \langle \varphi_1(\bar{x}), \dots, \varphi_\ell(\bar{x}) \rangle \quad (2.5)$$

is a valid formula that is satisfied by w iff

$$f(\langle \varphi_1(P_1), \dots, \varphi_\ell(P_1) \rangle, \dots, \langle \varphi_1(P_{n^k}), \dots, \varphi_\ell(P_{n^k}) \rangle) = 1.$$

The quantifier Q_f is called a *monoidal quantifier*. ▲

There are some immediate similarities between monoidal quantifiers and Lindström quantifiers. With light work, we see all monoidal quantifiers are equivalent to a Lindström

quantifier.

Theorem 2.2.1. *Fix an alphabet Σ with the corresponding logical vocabulary σ . Let $f = \{f_1, f_2, \dots\}$ be a monoidal function of arity k over a monoid M with parameters $\ell \in \mathbb{N}$, $T \subseteq M$, and $m : \{0, 1\}^\ell \rightarrow M$. Let Q_f be the corresponding monoidal quantifier, and consider the formula*

$$Q_f x_1, \dots, x_k : \langle \varphi_1(x_1, \dots, x_k), \dots, \varphi_\ell(x_1, \dots, x_k) \rangle \quad (2.6)$$

where each $\varphi_i \in \mathcal{L}(\sigma)$. Then there exists a language $L \subseteq M^*$ and a formula

$$Q_L x_1, \dots, x_k : \langle \psi_1(x_1, \dots, x_k), \dots, \psi_{s-1}(x_1, \dots, x_k) \rangle \quad (2.7)$$

that is logically equivalent to Equation (2.6), where each $\psi_i \in \mathcal{L}(\sigma)$. That is, every k -ary monoidal quantifier can be simulated by a k -ary Lindström quantifier.

Proof. As in Definitions 2.2.2, let $P_1, \dots, P_{n^k}$ be the elements of $\{1, \dots, n\}^k$ in lexicographic order, representing the consecutive positions of a word of length n^k . Let $\bar{b}_i \in \{0, 1\}^\ell$ for $1 \leq i \leq n^k$ be arbitrary ℓ -tuples. By Definition 2.2.1, $f(\bar{b}_1, \dots, \bar{b}_{n^k}) = 1$ if and only if the product $m(\bar{b}_1) \dots m(\bar{b}_{n^k})$ is in T . Observe that the latter condition is just another way of stating that the word $m(\bar{b}_1) \dots m(\bar{b}_{n^k}) \in M^*$ is in $\mathcal{WP}(M, T)$. Thus we can think of Q_f as mapping an ℓ -tuple of k -ary formulas to a word of length n^k over M^* . We need only to show that this mapping can be implemented by a first-order function, and thus by a Lindström quantifier over $L := \mathcal{WP}(M, T)$.

Let $M' = \{g_1, \dots, g_s\}$ be the image of the function m . We define $s - 1$ formulae $\psi_1, \dots, \psi_{s-1}$ sharing k free variables such that for all positions P_i , $\psi_j(P_i)$ is the first of these formulae to be true if and only if $m(\varphi_1(P_i), \dots, \varphi_\ell(P_i)) = g_j$. Since m is a finite function, we let

$$\psi_i(x_1, \dots, x_k) := \bigvee_{\langle b_1, \dots, b_\ell \rangle \in m^{-1}(g_i)} \langle \varphi_1(\bar{x}), \dots, \varphi_\ell(\bar{x}) \rangle = \langle b_1, \dots, b_\ell \rangle$$

for $i \in [s - 1]$. Given $w \in \Sigma^n$, if $a_1 \dots a_{n^k} \in M^*$ is the product resulting from Equation (2.6) in the manner previously described, then $w \models \psi_j(P_i)$ if and only if $a_i = g_i$. Thus if $w \models \psi_j(P_i)$, then we also have $w \not\models \psi_r(P_i)$ for $r \in [s - 1]$ not equal to j . It is easy to see from here that $\lambda\langle\psi_1, \dots, \psi_{s-1}\rangle(w) = a_1 \dots a_{n^k}$, which is exactly what we wanted to show. $\square$

Furthermore, we can achieve a partial converse to Theorem 2.2.1. Every Lindström quantifier over a regular language can be expressed by a monoidal quantifier over a finite monoid. Thus, finite monoidal quantifiers and regular Lindström quantifiers are equally powerful.

Theorem 2.2.2. *Fix an alphabet $\Sigma = \{a_1, \dots, a_s\}$ with the corresponding logical vocabulary σ . Let $L \in \mathbf{REG}$ be a regular language with Q_L the corresponding Lindström quantifier, and consider the formula*

$$Q_L x_1, \dots, x_k : \langle \varphi_1(x_1, \dots, x_k), \dots, \varphi_{s-1}(x_1, \dots, x_k) \rangle \quad (2.8)$$

where each $\varphi_i \in \mathcal{L}(\sigma)$. Then there exists a finite monoid M and a monoidal function $f = \{f_1, f_2, \dots\}$ over M such that for the monoidal quantifier Q_f , the formula

$$Q_f x_1, \dots, x_k : \langle \varphi_1(x_1, \dots, x_k), \dots, \varphi_{s-1}(x_1, \dots, x_k) \rangle \quad (2.9)$$

is logically equivalent to Equation (2.8).

Proof. Let $D = (Q, \Sigma, \delta, q_0, F)$ be any DFA recognizing L . Define the *transition monoid* of L to be $M_L := (\{\delta_w^* \mid w \in \Sigma^*\}, \circ, \delta_\varepsilon^*)$ where $\delta_w^* : Q \rightarrow Q$ is defined by $\delta_w^*(q) = \delta^*(q, w)$ for all $q \in Q$. (Here δ^* is the unique extension of δ to the domain Σ^* .) We will use this monoid $M := M_L$ to construct an appropriate monoidal. Note that M is finite.

The monoidal function $f = \{f_1, f_2, \dots\}$ will have the same arity k as Q_L . We will set the parameters $\ell = s - 1$, $T = \{\delta_w^* \in M \mid \delta^*(q_0, w) \in F\}$ and let $m : \{0, 1\}^{s-1} \rightarrow M$ be defined by $m(b_1, \dots, b_{s-1}) = \delta_{a_j}^*$ where j is the smallest integer such that $b_i = 1$, or $j = s$

otherwise. This completely defines f , and thus also defines the monoidal quantifier Q_f . It only remains to show that Equations (2.8) and (2.9) are logically equivalent.

Recall that the i -th letter of $\bar{w} := \lambda\langle\varphi_1, \dots, \varphi_{s-1}\rangle(w)$ is defined to be a_j if j is the smallest integer such that $w \models \varphi_j(P_i)$, and a_s if no such j exists. As we observed in Theorem 2.2.1, the monoidal quantifier Q_f can likewise be thought of as a map from tuples of formulae to strings; let $\bar{w}'$ be the string that w is mapped to under this action. Then $\bar{w}'_i$ is defined to be $m(\varphi_1(P_i), \dots, \varphi_{s-1}(P_i))$, which is equal to δ_j^* if j is the smallest integer such that $w \models \varphi_j(P_i)$ or δ_s^* if no such j exists. Therefore $\bar{w}' = \delta_{\bar{w}_1}^* \dots \delta_{\bar{w}_n}^*$. Evaluating $\bar{w}'$ as a monoid product in M , we obtain $\delta_{\bar{w}}^*$. Observe that $\delta_{\bar{w}}^* \in T$ if and only if $\delta^*(q_0, \bar{w}) \in F$ if and only if $\bar{w} \in L$, so (2.8) and (2.9) are indeed equivalent. $\square$

Corollary 2.2.2.1. *A language L is in $\mathbf{FO}(Q_f)$ (resp. $Q_f(\mathbf{FO})$) for some monoidal quantifier Q_f over a finite monoid if and only if there exists a regular language R such that L is in $\mathbf{FO}(Q_R)$ (resp. $Q_R(\mathbf{FO})$).*

2.2.2 Logical Characterizations of $\mathbf{NC}^1$ its Subclasses

We now proceed to analyze the **DLOGTIME**-uniform complexity classes $\mathbf{AC}^0 \subsetneq \mathbf{ACC}^0 \subseteq \mathbf{TC}^0 \subseteq \mathbf{NC}^1$ in this framework relating uniform circuit classes to first-order logics extended by Lindström quantifiers. All of the following results follow at least in part from Theorem 2.1.1, and are stated originally in [14].

At the lowest level, we have $\mathbf{AC}^0 = \mathbf{FO}_{\text{bit}}$. Letting $\mathcal{Z} = \{Q_{\mathcal{WP}(\mathbb{Z}/n\mathbb{Z})} \mid n \in \mathbb{N}\}$ be the set of modular counting quantifiers, we also have $\mathbf{ACC}^0 = \mathbf{FO}_{\text{bit}}(\mathcal{Z})$. Briefly skipping the class $\mathbf{TC}^0$, we finally observe that $\mathbf{NC}^1 = \mathbf{FO}_{\text{bit}}(Q_{\mathcal{WP}(S_5)}) = Q_{\mathcal{WP}(S_5)}(\mathbf{FO}_{\text{bit}})$ as a consequence of the **DLOGTIME**-uniform version of Barrington's Theorem in [14], which states that $\mathcal{WP}(S_5)$ is complete for uniform $\mathbf{NC}^1$ under uniform $\mathbf{AC}^0$ reductions. In fact, S_5 may be replaced by any non-solvable monoid, since this is a sufficient condition to apply the construction of Barrington's Theorem [2]. We note that this presentation of $\mathbf{ACC}^0$ in particular yields insight into its containment relation with $\mathbf{NC}^1$. Observing the result of Straubing *et al.* in [16] that quantifiers over solvable monoids can be simulated

by quantifiers in $\mathcal{Z}$, we have

Corollary 2.2.2.2. $NC^1 = ACC^0$ if and only if $NC^1 = FO_{bit}(\mathcal{Q})$ for some set of quantifiers $\mathcal{Q}$ over solvable monoids.

Returning to the matter of TC^0 , we now make the following observation. We have $TC^0 = FO_{bit}(Q_{MAJ})$, and indeed the unary majority quantifier Q_{MAJ} can be equivalently defined as a monoidal quantifier over the infinite monoid $(\mathbb{N} \times \mathbb{N}, +)$. It would be desirable to find a *finite* monoid (or regular language) which captures the power of majority so that we could attempt to separate TC^0 from its neighboring classes by algebraic means (e.g., via the theory of finite monoids). However such a monoid exists only if either $TC^0 = ACC^0$ or $TC^0 = NC^1$, as the monoid would either be solvable (yielding ACC^0) or non-solvable (yielding NC^1). Hence it is unlikely (or rather a daunting task to find) any monoidal quantifier over a finite monoid captures TC^0 . Under the hypothesis that $ACC^0 \subsetneq TC^0 \subsetneq NC^1$, we will need a different strategy to better understand the place of TC^0 in the first-order world; in this case TC^0 would contain all the solvable regular languages but none of them would capture the full difficulty of TC^0 .

To summarize, we have

Theorem 2.2.3 (First-Order Characterization of NC^1).

- $AC^0 = FO_{bit}$.
- $ACC^0 = FO_{bit}(\mathcal{Z})$ for $\mathcal{Z} = \{WP(\mathbb{Z}/n\mathbb{Z} \mid n \in \mathbb{N})\}$.
- $TC^0 = FO_{bit}(Q_{MAJ})$.
- $NC^1 = Q_{WP(M)}(FO_{bit}) = FO_{bit}(Q_{WP(M)})$ for all non-solvable monoids M .

As a side remark on uniformity and the power of BIT, it was observed by Barrington *et al.* that even in the absence of the BIT predicate, $FO(Q_{MAJ}^2) = TC^0$, where Q_{MAJ}^2 is the majority-of-pairs quantifier (i.e., has arity 2). They conjectured that the same does not hold for $FO(Q_{MAJ})$ without BIT, and such was confirmed by Lautemann *et al.* in their study of the more general class of *groupoidal quantifiers* [13]. We will see in

the subsequent section that passing into this more general picture will allow us to better understand the power of the BIT predicate and the relation between uniform circuit classes (up to **LOGCFL**) and first-order logics.

2.2.3 Monoidal Extensions of FO without BIT

Barrington *et al.* also devote considerable attention to **FO** classes without the BIT predicate. Since their methods require considerable algebraic machinery, we omit discussion of technical details, and instead simply restate the most important results. Most generally, we have

Theorem 2.2.4 (Theorem 10.6 [14]). *The languages expressible in **FO** with monoidal quantifiers over groups are precisely the regular languages.*

This raises an interesting observation about the relation between $\mathbf{TC}^0$ and $\mathbf{NC}^1$. We have seen that $\mathbf{NC}^1$ is expressible by group quantifiers with BIT, and thus removing BIT provably weakens the class. On the other hand, $\mathbf{FO}(Q_{\text{MAJ}}^2)$ yields the full class $\mathbf{TC}^0$ regardless of the presence of BIT.

A refinement of Theorem 2.2.4 yields a fine-grained classification of exactly which languages are expressible by certain group quantifiers.

Theorem 2.2.5 (Theorem 11.1 [14]). *Let $\mathcal{G}$ be a family of groups. A language L can be expressed using monoidal quantifiers over groups in $\mathcal{G}$ if and only if it is regular and L is recognized by a monoid M such that every simple group dividing it is a composition factor of a group in $\mathcal{G}$.*

Consequently, in the absence of BIT, quantifiers over non-solvable monoids do not even yield all of **REG**, let alone $\mathbf{NC}^1$, because there are infinitely many non-solvable simple groups.

Theorem 2.2.6. *Every finite monoid M witnesses $Q_M(\mathbf{FO}) \subsetneq \mathbf{REG}$.*

Furthermore, Theorem 2.2.5 allows us to easily place certain classes of regular languages in uniform circuit classes. For instance,

Theorem 2.2.7. *The solvable regular languages (i.e., those with solvable syntactic monoids) are contained in $\mathbf{ACC}^0$.*

Proof. Let $\mathcal{G}$ be the class of solvable monoids and let $L \in \mathbf{REG}$ have a solvable syntactic monoid $M(L)$ (recall the syntactic monoid of a regular language always recognizes it). Any divisor of a solvable monoid is solvable, so every simple group divisor of $M(L)$ is easily seen to be in $\mathcal{G}$. Since $\mathbf{ACC}^0 = \mathbf{FO}_{\text{bit}}(\mathcal{Z})$ can express the word problem over any solvable monoid [16], and in particular the word problems of groups in $\mathcal{G}$, then $L \in \mathbf{ACC}^0$ by Theorem 2.2.5. $\square$

A more detailed analysis of the regular languages (and more generally visibly counter languages) in $\mathbf{NC}^1$ will be conducted in Chapter 3.

2.3 Groupoidal Quantifiers and LOGCFL

2.3.1 Logical Characterizations of LOGCFL

In our study of first-order logics extended with monoidal quantifiers, we observed that if the monoid is finite, then a language is in the corresponding complexity class if it is reducible (either under $\leq_{\mathbf{FO}}$ or $\leq_{\mathbf{FO}}^t$ reductions) to an appropriate regular language. Since $\mathbf{REG} \subset \mathbf{NC}^1$ and there exist $\mathbf{NC}^1$ -hard regular languages, this descriptive approach offers a reasonable approach to elucidate the structure of $\mathbf{NC}^1$ in terms of regular languages and their monoids. A similar case presents itself in the class $\mathbf{LOGCFL}$. Lautemann, McKenzie, Schwentick, and Vollmer [13] introduced *groupoidal quantifiers* to initiate a descriptive theory of $\mathbf{LOGCFL}$ in the same vein as Barrington, Immerman, and Straubing. If we take Definitions 2.2.1 and 2.2.2 with groupoids instead of monoids to get *groupoidal quantifiers*, then a language is in the resulting extension of $\mathbf{FO}$ if it is reducible to an appropriate context-free language, since CFLs are precisely the word problems over finite groupoids (cf., e.g., [5]). Hence we arrive at the following definition.

Definition 2.3.1 (Groupoidal Quantifiers). If $L \in \mathbf{CFL}$, then the Lindström quantifier Q_L is called a *groupoidal quantifier*. $\blacktriangle$

Given that **LOGCFL** contains **CFL** and there exist **LOGCFL**-hard CFLs [5], we can expect the study of **FO** plus groupoidal quantifiers to yield insight into **LOGCFL** and its subclasses, similar to how **NC**¹ has been explored in [14]. The rest of this section reviews the results of Lautemann *et al.* [13] in this direction.

Our first theorem is the **LOGCFL**-analogue of the **NC**¹ result in Theorem 2.2.3.

Theorem 2.3.1 (Theorem 4.1 [13]). *There exists a groupoid G such that*

$$Q_G(\mathbf{FO}_{\text{bit}}) = \mathbf{FO}_{\text{bit}}(Q_{\text{Grp}}) = \mathbf{LOGCFL}.$$

Proof. The inclusion $Q_G(\mathbf{FO}_{\text{bit}}) \subseteq \mathbf{FO}_{\text{bit}}(Q_{\text{Grp}})$ is obvious. To see that $\mathbf{FO}_{\text{bit}}(Q_{\text{Grp}}) \subseteq \mathbf{LOGCFL}$, note that $L \in \mathbf{FO}_{\text{bit}}(Q_{\text{Grp}})$ iff there exists a **DLOGTIME**-uniform (hence L -uniform) **AC**⁰ Turing reduction from L to a set of groupoid word problems by Theorem 2.1.1. Expanding the AND gates into log-depth subcircuits and the oracle gates into **SAC**¹ circuits, and taking care of NOT gates by invoking the closure of **LOGCFL** under complement [6], we get a logspace-uniform **SAC**¹ circuit computing L , hence $L \in \mathbf{LOGCFL}$. Finally, we conclude $\mathbf{LOGCFL} \subseteq Q_G(\mathbf{FO}_{\text{bit}})$ by the result of [5], showing that there exists a CFL that is **LOGCFL**-complete under **DLOGTIME** reductions, and thus under **FO** reductions by Theorem 2.1.1. $\square$

Strikingly, **LOGCFL** can even be captured by a single groupoidal quantifier without the BIT predicate. This contrasts starkly with the case of $Q_M(\mathbf{FO})$ for a monoid M , where without the BIT predicate one cannot even express all the regular languages using a single monoidal quantifier (Theorem 2.2.6). Theorem 2.3.1 is thus strengthened to the following:

Theorem 2.3.2 (Theorem 4.2 [13]). *There exists a groupoid G such that*

$$\mathbf{FO}_{\text{bit}}(Q_{\text{Grp}}) = Q_G(\mathbf{FO}) = \mathbf{LOGCFL}$$

.

The proof of Theorem 2.3.2 is rather involved, so we merely offer an outline. Since

$\mathbf{LOGCFL} = \mathbf{FO}_{\text{bit}}(Q_{\text{Grp}})$, any language $A \in \mathbf{LOGCFL}$ can be expressed as $Q_L \bar{x} : \langle \Phi_1, \dots, \Phi_s \rangle$ where each Φ_i is a $\mathbf{FO}_{\text{bit}}$ formula. Using a monotone $\mathbf{FO}(Q_{\text{Grp}})$ formula which expresses the BIT predicate, each Φ_i can be transformed to a new specially structured formula Ψ_i which does not use the BIT predicate. A careful construction of a language L' depending on L allows us to replace the outer Lindström quantifier with $Q_{L'}$, and the formulae Ψ_i with quantifier-free formulae Ψ'_i , so that $Q_{L'} \bar{z} : \langle \Psi'_1, \dots, \Psi'_s \rangle$ is a $Q_{L'}(\mathbf{FO})$ formula expressing A . Since there is a homomorphism from any CFL to Greibach's hardest context-free language H , we can thus construct an equivalent $Q_{\text{pad}(H)}(\mathbf{FO})$ formula where $\text{pad}(H)$ is H with a neutral letter introduced. The (possibly non-unary) quantifier $Q_{\text{pad}(H)}$ applied to some (still quantifier-free) first-order formulae computes this homomorphism $L' \rightarrow H$ and thus expresses A .

Perhaps the most surprising consequence of Theorem 2.3.2 is that there exists a CFL complete for $\mathbf{LOGCFL}$ under quantifier-free projections.

Corollary 2.3.2.1 (Corollary 4.3 [13]). *Greibach's hardest context-free language augmented with a neutral letter is complete for $\mathbf{LOGCFL}$ under quantifier-free projections.*

Immerman and Landau in [11] suggested studying weak notions of reducibility such as first-order projections as a means of separating complexity classes. It would be interesting to see what, if anything, could be said about $\mathbf{LOGCFL}$ and its subclasses (such as $\mathbf{NC}^1$, which also has complete problems under projections) from this perspective.

2.3.2 Unary Groupoidal Extensions of FO without BIT

We've seen now that non-unary groupoidal quantifiers even without the BIT predicate can express all of $\mathbf{LOGCFL}$. What about *unary* groupoidal quantifiers without BIT?

Theorem 2.3.3 (Theorem 4.5 [13]). $Q_{\text{Grp}}^*(\mathbf{FO}) = \mathbf{CFL}$

Proof. The inclusion $\mathbf{CFL} \subseteq Q_{\text{Grp}}^*(\mathbf{FO})$ follows immediately from Definition 2.3.1. The reverse inclusion requires a technical lemma, namely that if $A \in Q_B^*(\mathbf{FO})$, then $A = h(R \cap g^{-1}(B))$ for homomorphisms h, g and a regular language D . The reader is referred

to Theorem 2.4 of [13] for a proof of this statement. Continuing with the argument, if $L \in Q_{\text{Grp}}^*(\mathbf{FO})$, then by definition $L \in Q_G^*(\mathbf{FO})$ for some finite groupoid G . Invoking the above lemma, $L = h(R \cap g^{-1}(\mathcal{WP}(G)))$ for some regular language R and homomorphisms g, h . Since **CFL** is closed under (inverse) homomorphism and regular intersection, and $\mathcal{WP}(G) \in \mathbf{CFL}$, we have $L \in \mathbf{CFL}$ as desired. $\square$

Naturally we may wonder if the same result holds for $Q_G^*(\mathbf{FO})$, for some specific groupoid G . In the monoidal world, we've seen that the analogous result fails as per Theorem 2.2.6. Indeed, the same is the case for groupoidal quantifiers.

Theorem 2.3.4 (Theorem 4.9 [13]). *Every finite groupoid G witnesses $Q_G^*(\mathbf{FO}) \subsetneq \mathbf{CFL}$.*

Proof. Suppose G is a finite groupoid such that $Q_G^*(\mathbf{FO}) = \mathbf{CFL}$. A k -limited PDA is a PDA whose stack alphabet contains at most k letters and pushes no more than k letters per transition. Every CFL is decided by some k -limited PDA, so let k_G be such that $\mathcal{WP}(G, \cdot)$ is recognized by a k_G -limited PDA. Lemma 4.8 of [13] states that the Dyck language over $t := (k_G + 1)^{k_G} + 1$ brackets is not recognizable by any k_G -limited PDA P . Using the finite automata characterization of unary first-order reductions from Theorem 3.5 of [13], we can construct a new PDA which computes the reduction from D_t to $\mathcal{WP}(G, \cdot)$ on-line while simultaneously using P to determine if the result is in $\mathcal{WP}(G, \cdot)$, contradicting the aforementioned lemma. $\square$

Elementary facts about **CFL** (non-closure under intersection, complement) also yield the following hierarchy. Note that no such theorem was given in the regular case, due to the rather strong closure properties of **REG**.

Corollary 2.3.4.1 (Corollary 4.6 [13]).

$$\begin{aligned} Q_{\text{Grp}}^*(\mathbf{FO}) &= \mathbf{CFL} \subsetneq BC^+(Q_{\text{Grp}}^*(\mathbf{FO})) = BC^+(\mathbf{CFL}) \\ &\subsetneq BC(Q_{\text{Grp}}^*(\mathbf{FO})) = BC(\mathbf{CFL}) \\ &\subseteq \mathbf{FO}(Q_{\text{Grp}}^*) \end{aligned}$$

2.3.3 Unary Groupoidal Extensions of FO with BIT

Lautemann *et al.* raise the question of the relation between the classes

$$Q_{\text{Grp}}^*(\mathbf{FO}) \subseteq Q_{\text{Grp}}^*(\mathbf{FO}_{\text{bit}}) \subseteq \mathbf{FO}_{\text{bit}}(Q_{\text{Grp}}^*) \subseteq \mathbf{LOGCFL},$$

i.e. the first-order logics extended by unary groupoidal quantifiers. One may expect this hierarchy to be strict, but proving so would amount to a breakthrough in circuit lower bounds.

Lemma 2.3.5 (Lemma 4.12 [13]). $\mathbf{TC}^0 \subseteq \mathbf{FO}_{\text{bit}}(Q_{\text{Grp}}^*)$.

Proof. The class $\mathbf{TC}^0$ is precisely $\mathbf{FO}_{\text{bit}}(Q_{\text{MAJ}}^*)$ as per Theorem 2.2.3, and $\text{MAJ} \in \mathbf{CFL}$.

□

Hence proving $Q_{\text{Grp}}^*(\mathbf{FO}_{\text{bit}}) \subsetneq \mathbf{LOGCFL}$ is at least as hard as showing $\mathbf{TC}^0 \subsetneq \mathbf{LOGCFL}$.

While proving separations between classes with the BIT predicate may be prohibitively difficult with present methods, Lautemann *et al.* were able to show that $\mathbf{FO}_{\text{bit}}(Q_{\text{Grp}}^*)$ is strictly weaker when the BIT predicate is removed. This can be seen as a more general case of the result that $Q_{\text{Grp}}^*(\mathbf{FO}) = \mathbf{CFL} \subsetneq Q_{\text{Grp}}^*(\mathbf{FO}_{\text{bit}})$, which is witnessed, for instance, by the set of strings whose lengths are powers of 2.

Theorem 2.3.6 (Theorem 4.16 [13]). $\mathbf{FO}(Q_{\text{Grp}}^*)$ is strictly contained in $\mathbf{FO}_{\text{bit}}(Q_{\text{Grp}}^*)$.

Their proof employs an intricate combinatorial argument to show that any set in $\mathbf{FO}(Q_{\text{Grp}}^*) \cap 0^*$ is semi-linear, and thus in $\mathbf{FO}[+]$, so that the language $\{0^{n^2} \mid n \geq 0\} \in \mathbf{FO}_{\text{bit}}(Q_{\text{Grp}}^*)$ separates the two classes. As we mentioned in Section 2, a corollary of Theorem 2.3.6 affirms a conjecture of Barrington *et al.* in [14].

Corollary 2.3.6.1 (Corollary 4.17 [13]). *The BIT predicate cannot be expressed in $\mathbf{FO}(Q_{\text{MAJ}})$.*

2.4 The First-Order Landscape of LOGCFL and Open Problems

The main open problems identified in Chapter 2 and some more are listed below.

1. What can be said about classes defined by unary monoidal quantifiers? We did not pay careful attention to the detail of arity in our analysis of monoidal quantifiers, but open problems along these lines are likely to be tractable because of the powerful tools from the theory of finite monoids at our disposal. Many results are likely “laying around” in the details of proofs.
2. We’ve seen that $Q_{\text{Grp}}^*(\mathbf{FO}) = \mathbf{CFL}$, and it is easy to see $Q_{\text{Mon}}^*(\mathbf{FO}) = \mathbf{REG}$ in the same manner. Does the class of visible counter languages $\mathbf{VCL}$ permit such a characterization?
3. How does $Q_{\text{Grp}}^*(\mathbf{FO}_{\text{bit}})$ relate to $\mathbf{TC}^0$? If $\mathbf{TC}^0 \not\subseteq Q_{\text{Grp}}^*(\mathbf{FO}_{\text{bit}})$, can we show that $Q_{\text{Grp}}^*(\mathbf{FO}_{\text{bit}})$ is strictly contained in $\mathbf{LOGCFL}$?
4. In the groupoidal setting, we know $\mathbf{FO}(Q_{\text{Grp}}^*) \subsetneq \mathbf{FO}_{\text{bit}}(Q_{\text{Grp}}^*)$. Is it also the case that $\mathbf{FO}(Q_{\text{Mon}}^*) \subsetneq \mathbf{FO}_{\text{bit}}(Q_{\text{Mon}}^*)$?
5. Can we characterize (for instance, algebraically) a subset of the finite groupoids G such that $\mathbf{LOGCFL} = Q_G(\mathbf{FO})$, or at the very least identify essentially new groupoids for which the result holds?
6. Does $Q_G^*(\mathbf{FO})$ fail to capture all regular languages as it fails to capture all context-free languages? We know from the theory of monoids that $Q_M^*(\mathbf{FO}) \subsetneq \mathbf{REG}$ for all finite monoids M (Theorem 2.2.6).
7. Is $\mathbf{BC}(\mathbf{CFL})$ strictly contained in $\mathbf{FO}(Q_{\text{Grp}}^*)$? Gaining a better understanding of $\mathbf{BC}(\mathbf{CFL})$ seems to be a prerequisite to classifying the context-free languages in $\mathbf{NC}^1$ and its subclasses in a systematic fashion, since languages in circuit classes are closed under boolean operations.

Chapter 3

The Complexity of Some Formal Language Families

3.1 Regular Languages

In this section we are concerned with the relative complexity of regular languages. We want to show that a regular language is at least as hard as the monoids “contained in it.” The following lemma of Barrington, Compton, Straubing, and Theri  n establishes a more general result:

Lemma 3.1.1 (Regular Language Reduction Lemma [4]). *Let $L_1, L_2 \in \mathbf{REG}$ be regular languages over a finite alphabet Σ . Suppose that L_1 is recognized by a homomorphism $\delta : \Sigma^* \rightarrow N$, where N is a divisor of $\mathcal{M}(L_2)$ via a surjective homomorphism $\pi : M \rightarrow N$ for some $M \subseteq \mathcal{M}(L_2)$. Suppose further that there exists an integer $t > 0$ such that for all $a \in \Sigma$, there exists $w \in \Sigma^t$ such that $\pi(\eta_{L_2}(w)) = \delta(a)$. Then L_1 reduces to L_2 via $\mathbf{NC}^0$ Turing reductions.*

Proof. For all $u, v \in \Sigma^*$, we define the language

$$u^{-1}L_2v^{-1} := \{w \in \Sigma^* \mid uwv \in L_2\}. \quad (3.1)$$

Observe that such languages are a union of equivalence classes of $\mathcal{M}(L_2)$: if $\eta_{L_2}(w) = \eta_{L_2}(w')$ then $w \in u^{-1}L_2v^{-1} \iff w' \in u^{-1}L_2v^{-1}$, so $w \in u^{-1}L_2v^{-1}$ implies $\eta_{L_2}(w) \subseteq u^{-1}L_2v^{-1}$. By hypothesis L_2 is regular, hence $\mathcal{M}(L_2)$ is finite, so (3.1) defines only finitely many languages. Let $u_1, v_1, \dots, u_k, v_k \in \Sigma^*$ be such that $\{u_i^{-1}L_2v_i^{-1} \mid i \in [k]\}$ is the set of all of these languages.

We now make the critical observation that for all $w \in \Sigma^*$, the k -vector of truth values

$$\langle w \in u_1^{-1}L_2v_1^{-1}, \dots, w \in u_k^{-1}L_2v_k^{-1} \rangle \in \{0, 1\}^k \quad (3.2)$$

completely determines the value of $\eta_{L_2}(w)$. Formally, if ρ is the map sending words to the k -vector defined in (3.2), then there exists a finite function $\phi : \{0, 1\}^k \rightarrow \mathcal{M}(L_2)$ such that $\eta_{L_2} = \phi \circ \rho$. We now prove this fact. Let $w \in \Sigma^*$ and suppose $\rho(w) = \langle b_1, \dots, b_k \rangle$. We claim that

$$\eta_{L_2}(w) = \bigcap_{[i \in [k]; b_i=1]} u_i^{-1}L_2v_i^{-1}. \quad (3.3)$$

Let $i_1, \dots, i_r \in [k]$ be the maximum list of indices such that $\eta_{L_2}(w) \subseteq u_{i_1}^{-1}L_2v_{i_1}^{-1}, \dots, u_{i_r}^{-1}L_2v_{i_r}^{-1}$. For every equivalence class $\eta_{L_2}(w') \in \mathcal{M}(L_2) \setminus \eta_{L_2}(w)$, there exists some index i_j such that $\eta_{L_2}(w) \in u_{i_j}^{-1}L_2v_{i_j}^{-1}$, but $\eta_{L_2}(w') \notin u_{i_j}^{-1}L_2v_{i_j}^{-1}$. This is because there exist $u, v \in \Sigma^*$ such that $uwv \in L_2$ and $uw'v \notin L_2$ or vice-versa by the definition of L_2 -congruence. Hence $u^{-1}L_2v^{-1}$ has the desired properties, and it must be equal to some $u_{i_j}^{-1}L_2v_{i_j}^{-1}$ by hypothesis. The equality in (3.3) now follows immediately, and with it the existence of ϕ .

We will now describe an $\mathbf{NC}^0[L_2]$ circuit with n inputs that decides L_1 . By hypothesis, for each $a \in \Sigma$, there is a word $w_a \in \Sigma^t$ such that $\pi(\eta_{L_2}(w_a)) = \delta(a)$. Hence we make the following computations:

1. Compute $x_i \mapsto w_{x_i}$ for all $i \in [2, n-1]$.
2. Compute $x_1 \mapsto u_i w_{x_1}$ and $x_n \mapsto w_{x_n} v_i$ for all $i \in [k]$.
3. Compute $\rho(w_{x_1} \dots w_{x_n})$ by computing the truth value of $u_i w_{x_1} \dots w_{x_n} v_i \in L_2$ for all $i \in [k]$ with an L_2 gate, where the i -th component takes $nt + |u_i v_i|$ inputs. We subtly

use the fact that w_{x_i} has length t in this construction; otherwise we would not know how many boolean arguments the sub-circuit needs to take in as inputs.

4. Compute ϕ on $\rho(w_{x_1} \dots w_{x_n})$ to get $\eta_{L_2}(w_{x_1} \dots w_{x_n})$. The output is some reasonable encoding of an element of $\mathcal{M}(L_2)$.
5. Compute π on $\phi(\rho(w_{x_1} \dots w_{x_n})) = \eta_{L_2}(w_{x_1} \dots w_{x_n})$ and test whether the result is in the accepting subset of N . This is the output of the circuit.

For the proof of correctness, observe that

$$\begin{aligned} \pi(\eta_{L_2}(w_{x_1} \dots w_{x_n})) &= \pi(\eta_{L_2}(w_{x_1}) \dots \eta_{L_2}(w_{x_n})) \\ &= \pi(\eta_{L_2}(w_{x_1})) \dots \pi(\eta_{L_2}(w_{x_n})) \\ &= \delta(x_1) \dots \delta(x_n). \end{aligned}$$

Hence Step 5 is essentially checking whether $\delta(x_1) \dots \delta(x_n)$ is in the accepting subset of N , which is equivalent to checking if $x_1 \dots x_n \in L_1$. It is easy to verify that each step only requires constant depth, and in fact linear size, and furthermore that each non-oracle gate has constant fan-in. Hence $L_1 \leq_{\mathbf{NC}^0}^t L_2$. $\square$

We remark that the key “trick” involved in the proof of Lemma 3.1.1 is to compute $\eta_{L_2}(w_{x_1} \dots w_{x_n})$ directly, rather than element-wise which yields a linear-size monoid product.

The regular language reduction lemma thus motivates the following definition, which aims to capture the set of monoids that are “contained in” a regular language.

Definition 3.1.1 (Monoids of a Regular Language). Let $L \in \mathbf{REG}$ be a regular language over an alphabet Σ . Then $\mathcal{D}(L)$ is the set of simple monoids N such that there exists an integer $t > 0$ and a set $W \subseteq \Sigma^t$ such that N divides $\eta_L(W)$. $\blacktriangle$

Note that the condition that there exists a set $W \subseteq \Sigma^t$ such that N divides $\eta_L(W)$ is essentially the same as the demand of Lemma 3.1.1 that for all $a \in \Sigma$, there exists $w \in \Sigma^t$ such that $\pi(\eta_{L_2}(w)) = \delta(a)$. This easily yields the following theorem:

Corollary 3.1.1.1. *Let $L \in \mathbf{REG}$ be a regular language over an alphabet Σ . Then for all $N \in \mathcal{D}(L)$ and $T \subseteq N$, we have $\mathcal{WP}(N, T) \leq_{\mathbf{NC}^0}^t L$. That is, word problems over monoids in $\mathcal{D}(L)$ are $\mathbf{NC}^0$ Turing-reducible to L .*

Proof. The language $\mathcal{WP}(N, T)$ is clearly recognized by a homomorphism $\delta : N^* \rightarrow N$. Since $N \in \mathcal{D}(L)$, there is some integer $t > 0$ and subset $W \subseteq \Sigma^t$ such that N divides $\eta_L(W)$. In particular, N must also divide $\eta_L(\Sigma^*) = \mathcal{M}(L)$ by the definition of monoid division. Furthermore, there exists a surjective homomorphism $\pi : \eta_L(W) \rightarrow N$, and for all $a \in N$, there is some word $w \in W$ such that $\pi(\eta_L(w)) = a$ by the surjectivity of π and $|w| = t$ by the definition of W . Hence the conditions of Lemma 3.1.1 are met, and $\mathcal{WP}(N, T) \leq_{\mathbf{NC}^0}^t L$ as desired. $\square$

We can also make a general statement about the regular languages in constant-depth circuit classes in terms of $\mathcal{D}(L)$:

Theorem 3.1.2. *Let $\mathcal{C}$ be a constant-depth circuit class, i.e. $\mathcal{C} = \mathbf{AC}^0[L_1, \dots, L_n]$ for some $L_1, \dots, L_n \subseteq \{0, 1\}^*$. Then a regular language L is in $\mathbf{REG} \cap \mathcal{C}$ if and only if for all $N \in \mathcal{D}(L)$, the word problem $\mathcal{WP}(N, \cdot)$ is in $\mathcal{C}$.*

Proof. Suppose $L \in \mathbf{REG} \cap \mathcal{C}$. We have $\mathcal{WP}(N, \cdot) \leq_{\mathbf{NC}^0}^t L$ for $N \in \mathcal{D}(L)$ by Lemma 3.1.1.1. Hence $\mathcal{WP}(N, \cdot) \in \mathcal{C}$, since the reduction can be computed in $\mathcal{C}$. Conversely, suppose $\mathcal{WP}(N, \cdot) \in \mathcal{C}$ for all $N \in \mathcal{D}(L)$. It follows from Theorem 2.2.5 that L is expressible in $\mathbf{FO}$ with $\mathcal{D}(L)$ quantifiers, and by Theorem 2.1.1 it is thus expressible in $\mathcal{C}$. $\square$

The main results of this section concern the regular languages in $\mathbf{AC}^0$ and $\mathbf{ACC}^0[p]$ for a prime p . As a consequence of Lemma 3.1.1 and the circuit lower bounds of [8], [15], we obtain the following two theorems.¹

Theorem 3.1.3. *Let $L \in \mathbf{REG}$ be a regular language over an alphabet $\{0, 1\}$. Then $L \in \mathbf{AC}^0$ if and only if $\mathcal{D}(L)$ contains no nontrivial group.*

¹These results originally appeared in [4], and later in [17]; the proofs here are partially drawn from [4].

Proof. Suppose that $L \in \mathbf{AC}^0$. For the sake of contradiction assume there exists an integer $t \geq 0$ such that $\eta_L(\{0,1\}^t)$ contains a nontrivial group G , i.e. $G \in \mathcal{D}(L)$. Let $g \in G$ be an element of order $k > 0$, and let $G' = \langle g \rangle$ be the subgroup generated by g . Choose $u, v \in \{0,1\}^t$ such that $\eta_L(u) = g^k = 1_G$ and $\eta_L(v) = g$ (u, v exist by hypothesis). Finally, define a homomorphism $\delta : \{0,1\}^* \rightarrow G'$ by $\delta(0) = g^k = 1_G$ and $\delta(1) = g$. Then $\eta_L(u) = \delta(0)$ and $\eta_L(v) = \delta(1)$. Applying Lemma 3.1.1, the language $\text{MOD}_k = \{w \in \{0,1\}^* \mid |w|_1 \equiv 0 \pmod{k}\}$ is reducible to L , and thus in $\mathbf{AC}^0$. This contradicts the result of [8] that $\text{MOD}_k \notin \mathbf{AC}^0$, so $G \notin \mathcal{D}(L)$.

Conversely, suppose that $\mathcal{D}(L)$ contains no nontrivial groups. In particular it contains no nontrivial simple groups, so $\mathcal{D}(L)$ contains at most the aperiodic unit U_1 . Since $\mathcal{WP}(U_1, \cdot) \in \mathbf{AC}^0$, it follows by Theorem 3.1.2 that $L \in \mathbf{AC}^0$. $\square$

Theorem 3.1.4. *Let $L \in \mathbf{REG}$ be a regular language over the alphabet Σ , and let p be a prime. Then $L \in \mathbf{ACC}^0[p]$ if and only if the only groups in $\mathcal{D}(L)$ are p -groups.*

Proof. The proof is analogous to that of 3.1.3; the main difference is that it uses the fact that $\text{MOD}_q \notin \mathbf{ACC}^0[p]$ for p, q distinct primes [15], and that word problems over p -groups are in $\mathbf{ACC}^0[p]$ [16]. $\square$

3.2 Visibly Counter Languages

In Chapters 2 and 3.1 we observed that it is very convenient to view problems of circuit complexity as questions about regular languages and first-order reducibility. However this approach falls short in some regards; for instance, regular languages fail to characterize $\mathbf{TC}^0$ as they characterize $\mathbf{AC}^0$, $\mathbf{ACC}^0$, and $\mathbf{NC}^1$ in Theorem 2.2.3. In [9], Hahn, Krebs, Lange, and Ludwig continued this program of studying the intersection of circuit classes with formal language families by investigating the first-order complexity of visibly counter languages. The class $\mathbf{VCL}$ retains many of the same closure properties of $\mathbf{REG}$ while possessing strictly less computational power than $\mathbf{CFL}$ [1], which makes it a natural next step in understanding the relationship between automata and circuits.

First we outline the results of Krebs, Lange, and Ludwig in [12] who showed how to decompose visibly counter automata into a “stack part” and a “regular part”. They investigated the visibly counter languages in $\mathbf{AC}^0$ and essentially proved that the circuit complexity of a particular VCL depends on how complex stack operations are, and what algebraic properties the regular part possesses. Afterwards, we consider their joint work with Hahn in [9], which lifts the results of Barrington *et al.* in [4] from $\mathbf{REG}$ to its superclass $\mathbf{VCL}$.

3.2.1 Height Computation

Let $\mathcal{A}$ be a VCA over a visibly pushdown alphabet $\hat{\Sigma}$ and let $w \in \hat{\Sigma}^*$. A sensible approach to check the membership of w in $\mathcal{L}(\mathcal{A})$ is to compute, for each position $i \in [|w|]$, the height $\Delta(w_1 \dots w_i)$, and consult the transition rules for $\mathcal{A}$ to determine the configuration yielded by $w_1 \dots w_{i+1}$. Hence we concern ourselves with the descriptive complexity of defining a height predicate $H_n(\cdot)$ for each $n \in \mathbb{N}$ such that $H_n(i)$ is true in w iff $\Delta(w_1 \dots w_i) = n$. In general, $\mathbf{FO}(\text{MAJ})$ is sufficient for arbitrary height computation. Yet if the computation of $\mathcal{A}$ can be split into segments during which the stack height behaves linearly, H_n turns out to be quite simple (in fact, in $\mathbf{FO}[+] \subset \mathbf{AC}^0$). The main achievement of Krebs *et al.* in [12] along these lines was a classification of when H_n is definable in $\mathbf{FO}[+]$. The following definition formally captures this linear height behavior.

Definition 3.2.1 (Fixed Slope). Let $\mathcal{A}$ be an m -VCA with state set Q and input alphabet Σ . We say that a state $q \in Q$ has a *fixed slope* if there exists a slope $\alpha \in \mathbb{Q}$ and a corridor $\gamma \in \mathbb{N}$ so that for all $w \in \Sigma^*$, if $(q, h_1) \xrightarrow{w} (q, h_2)$ and $h_1 + \Delta(w') \geq m$ for all prefixes w' of w , then the following conditions hold:

- $h_2 = \alpha|w| + h_1$.
- $|\Delta(w') - \alpha|w'|| \leq \gamma$.

▲

However, even if some states do not have a fixed slope, it is still possible that H_n may be expressed in $\mathbf{FO}[+]$. This is the case if for all relevant inputs $w \in \hat{\Sigma}$ such that a prefix w' of w takes $\mathcal{A}$ to a state $q \in Q$ without a fixed slope, the stack height of $\mathcal{A}$ no longer matters after reading w' .

Definition 3.2.2 (Active State). Let $\mathcal{A}$ be an m -VCA with state set Q . We say that a state $q \in Q$ is *active* if there is a word $w \in \mathcal{L}(\mathcal{A})$ and positions i, j ($i < j$) such that $\mathcal{A}$ on input $w_1 \dots w_i$ reaches q , $\Delta(w_1 \dots w_i) > m + |Q|$, and $\Delta(w_1 \dots w_i) - \Delta(w_1 \dots w_j) > |Q|$. $\blacktriangle$

Intuitively, a state $q \in Q$ is active when some $w \in \mathcal{L}(\mathcal{A})$ takes $\mathcal{A}$ above the height threshold m , and eventually dips back down below m . If the stack height stays above m for the rest of the computation, then it is irrelevant as $\mathcal{A}$ essentially behaves as a finite automaton. Our observations hitherto yield the following definition:

Definition 3.2.3 (Simple Height Behavior). Let $\mathcal{A}$ be an m -VCA with state set Q . We say that $\mathcal{A}$ has *simple height behavior* if every active state in Q also has a fixed slope. $\blacktriangle$

As the following lemmas show, simple height behavior is a sufficient condition for (a variant of) H_n to be expressible in $\mathbf{FO}[+]$, and when simple height behavior is violated, the stack height behavior is essentially as complex as possible (i.e., it is $\mathbf{TC}^0$ -hard and thus requires general counting).

Lemma 3.2.1 (Lemma 22 [12]). *Let $L \in \mathbf{VCL}$ be a visibly counter language over a visibly pushdown alphabet $\hat{\Sigma}$. If L is recognized by an m -VCA with simple height behavior, then for each $n < m$ and $w \in \hat{\Sigma}^*$ and $i \in [|w|]$, we can define an $\mathbf{FO}[+]$ predicate $H'_n(\cdot)$ such that*

- *If $w \notin L$, then $w \models H'_n(i)$ only if $\Delta(w_1 \dots w_{i-1}) = k$.*
- *If $w \in L$, then $w \models H'_n(i)$ if and only if $\Delta(w_1 \dots w_{i-1}) = k$.*

For heights above m , we define $H'_{\geq m}(x) \equiv \neg \bigvee_{i=0}^{m-1} H'_i(x)$. Although the H'_n predicates for $n < m$ permit false negatives when $w \notin L$, and $H'_{\geq m}$ permits false positives, these predicates are nevertheless powerful enough to prove later theorems, such as the following hardness result:

Lemma 3.2.2 (Lemma 5 [9]). *Let $L \in \mathbf{VCL}$ be a visibly counter language. If L does not have a VCA with simple height behavior, then L is $\mathbf{TC}^0$ -hard under $\mathbf{NC}^0$ Turing reductions.*

As a consequence of this lemma, we see that simple height behavior is in fact a property of languages, even though it is defined at first as a property of automata. We hereafter understand simple height behavior to apply to languages in $\mathbf{VCL}$.

3.2.2 Regular Part Computation

Having handled already the complexity of computing the stack behavior, we now analyze the complexity of computing the finite state control of $\mathcal{A}$. With some care, we can, in effect, reduce this problem to one about regular languages, which we understand very well.

Definition 3.2.4 (Height Transduction). Let Σ be a visibly counter alphabet and let $w \in \Sigma^*$. We define a new alphabet $\Sigma_m := \Sigma \times [m]$ and the m -height of w to be $\Delta_m(w) := \min(\Delta(w), m)$ for all $m \in \mathbb{N}$. The m -height transduction of w is given by the function $\tau_m : \{w \in \Sigma^* \mid \Delta(w) \geq 0\} \rightarrow \Sigma_m^*$ where

$$\tau_m(w_1 \dots w_n) = (w_1, \Delta_m(\varepsilon))(w_2, \Delta_m(w_1)) \dots (w_n, \Delta_m(w_1 \dots w_{n-1})).$$

▲

Given a VCL L decided by some m -VCA $\mathcal{A}$, we can consider the finite state control of $\mathcal{A}$ separately from its counter by pre-computing the height profile of words with the m -height transduction τ_m . This defines a regular language which we call the *regular part* of L .

Definition 3.2.5 (Regular Part of a VCL). Let $L \in \mathbf{VCL}$ be a visibly counter language and let $\mathcal{A} = (Q, q_0, F, \hat{\Sigma}, \delta_0, \dots, \delta_m)$ be an m -VCA such that $\mathcal{L}(\mathcal{A}) = L$. Define the NFA $M = (Q, q_0, F, \Sigma_m, \delta)$ where $\delta(q, (a, i)) = \delta_i(q, a)$. The *regular part* of $\mathcal{A}$ is the language $R_{\mathcal{A}} := \mathcal{L}(M)$.

▲

Clearly $w \in L \iff \tau_m(w) \in R_{\mathcal{A}}$. However we must note that for a VCL L and a corresponding m -VCA $\mathcal{A}$, the regular part $R_{\mathcal{A}}$ contains words not in $\tau_m(\Sigma^*)$; in particular $R_{\mathcal{A}} \cap \overline{\tau_m(\Sigma^*)}$ varies depending on the choice of $\mathcal{A}$, and some theorems of [12, 9] (which we later state) are only known to go through when a certain *loop-normal form* is imposed on $\mathcal{A}$. Fortunately for every VCA there exists an equivalent VCA in loop-normal form. The reader is referred to the aforementioned papers for a definition of this restriction; we do not find it particularly insightful to state here, outside the context of the more detailed discussion offered by Krebs and Hahn *et al.*

3.2.3 Hardness Results

In order to state a reduction lemma for visibly counter languages as we did for regular languages in Lemma 3.1.1, we define the simple monoids of a visibly counter language:

Definition 3.2.6 (Simple Monoids of a VCL). Let $L \in \mathbf{VCL}$ be a visibly counter language over an alphabet Σ and let $\mathcal{A}$ be an m -VCA in loop-normal form recognizing L . Then $\mathcal{D}(L)$ is the set of simple monoids N such that there exists an integer $t > 0$ and a set $W \subseteq \Sigma_m^t$ with $W^* \subseteq F(\tau_m(\Sigma^*))$ so that N divides $\eta_{R_{\mathcal{A}}}(W)$. ▲

This is much the same as Definition 3.1.1, with the addition that we take care to account for words over Σ_m whose height component represents an illegal computation (i.e., one that does not agree with τ_m). We then have:

Lemma 3.2.3 (VCL Reduction Lemma [9]). *Let $L \in \mathbf{VCL}$ be a visibly counter language. Then for all $N \in \mathcal{D}(L)$ and $T \subseteq N$, we have $\mathcal{WP}(N, T) \leq_{\mathbf{NC}^0}^t L$.*

Hahn *et al.* also give a complementary upper bound on visibly counter languages in terms of $\mathcal{D}(L)$. Regular predicates and monoidal quantifiers over monoids in $\mathcal{D}(L)$ suffice to express the subset of legal words in the regular part of a VCL L :

Lemma 3.2.4 (Lemma 14 [9]). *Let $L \in \mathbf{VCL}$ be a visibly counter language over an alphabet Σ and let $\mathcal{A}$ be an m -VCA recognizing L . Then there is an $\mathbf{FO}(Q_L)[\text{Reg}]$ formula*

φ such that

$$\mathcal{L}(\varphi) \cap \tau_m(\Sigma^*) = R_{\mathcal{A}} \cap \tau_m(\Sigma^*)$$

As a consequence of Lemma 3.2.3 and Lemma 3.2.4 with Theorem 2.1.1, the following completeness results for **VCL** follow:

Theorem 3.2.5 (VCL Main Theorem [9]). *Let $L \in \mathbf{VCL}$ be a visibly counter language. Then*

- *If $L \notin \mathbf{REG}$ and L has simple height behavior and $\mathcal{D}(L)$ does not contain a simple group, then $L \in \mathbf{AC}^0$.*
- *If $L \in \mathbf{REG}$ and $\mathcal{D}(L)$ contains only simple cyclic groups $\mathbb{Z}_{p_1}, \dots, \mathbb{Z}_{p_k}$, then L is $\mathbf{CC}^0[p_1, \dots, p_k]$ -complete.*
- *If $L \notin \mathbf{REG}$ and L has simple height behavior and $\mathcal{D}(L)$ contains only U_1 and simple cyclic groups $\mathbb{Z}_{p_1}, \dots, \mathbb{Z}_{p_k}$, then L is $\mathbf{ACC}^0[p_1, \dots, p_k]$ -complete.*
- *If L does not have simple height behavior and $\mathcal{D}(L)$ does not contain a non-solvable group, then L is $\mathbf{TC}^0$ -complete.*
- *If $\mathcal{D}(L)$ contains a nonsolvable group, then L is $\mathbf{NC}^1$ -complete.*

where completeness is with respect to $\mathbf{AC}^0$ Turing reductions.

In contrast with Theorem 2.2.3, Theorem 3.2.5 describes a family of formal languages that are complete for $\mathbf{TC}^0$. It tells us, for instance, that $\mathbf{ACC}^0 \neq \mathbf{TC}^0$ iff $\mathbf{ACC}^0 \cap \mathbf{VCL} = \mathbf{TC}^0 \cap \mathbf{VCL}$. This suggests that further investigation of pushdown automata with restricted stacks may be a useful avenue to better understand $\mathbf{TC}^0$.

Chapter 4

Conclusion

In this thesis we surveyed two aspects of the connection between automata, circuits, and logic. On the one hand, we studied the characterization of the uniform circuit classes $\mathbf{AC}^0 \subset \mathbf{ACC}^0 \subseteq \mathbf{TC}^0 \subseteq \mathbf{NC}^1 \subseteq \mathbf{LOGCFL}$ in terms of first-order logics augmented by Lindström quantifiers. In other words, we presented each of these classes as a set of languages first-order reducible to some fixed language (usually regular or context-free). On the other hand, we parameterized the circuit complexity of regular and visibly counter languages by their algebraic properties. The complexity of a regular language is determined by the complexity of the monoids dividing its syntactic monoid modulo a technical condition. The complexity of a visibly counter language is determined by complexity of its so-called regular part and by the complexity of its counter behavior.

Of the open problems and directions for further research, we emphasize the following. First, the investigation of first-order classes defined by monoidal quantifiers initiated by Barrington *et al.* in [14] should be conducted more carefully using the Lindström quantifier formalism that we have discussed in detail in Chapter 2 of this thesis. Problems of quantifier arity and nestedness have not yet been seriously considered in the monoidal setting; it would be interesting to see if we can answer such questions especially where we do not have answers in the groupoidal setting. Second, it would be desirable to produce some concrete visibly counter languages satisfying the various completeness criteria of Theorem

3.2.5. Third, we should identify where our notions of reduction can be strengthened; for instance, which of our reducibility relations still hold under (quantifier-free) first-order projections? All of the reductions we've constructed in this thesis are surely at least first-order reductions; it should not be difficult to show this explicitly.

Bibliography

- [1] Vince Bárány, Christof Löding, and Olivier Serre. “Regularity Problems for Visibly Pushdown Languages”. In: *STACS 2006*. Ed. by Bruno Durand and Wolfgang Thomas. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 420–431.
- [2] David A. Barrington. “Bounded-width polynomial-size branching programs recognize exactly those languages in NC1”. In: *Journal of Computer and System Sciences* 38.1 (1989), pp. 150–164.
- [3] David A. Mix Barrington and Denis Thérien. “Finite monoids and the fine structure of NC1”. In: *J. ACM* 35.4 (Oct. 1988), pp. 941–952.
- [4] David A. Mix Barrington and Denis Thérien. “Finite monoids and the fine structure of NC1”. In: *J. ACM* 35 (1988), pp. 941–952.
- [5] François Bédard, François Lemieux, and Pierre McKenzie. “Extensions to Barrington’s M-program model”. In: *Theoretical Computer Science* 107.1 (1993), pp. 31–61.
- [6] Allan Borodin, Stephen A. Cook, Patrick W. Dymond, Walter L. Ruzzo, and Martin Tompa. “Two Applications of Inductive Counting for Complementation Problems”. In: *SIAM J. Comput.* 18.3 (1989), pp. 559–578.
- [7] “First Order Predicate Logic with Generalized Quantifiers”. In: *Theoria* 32.3 (1966), pp. 186–195.
- [8] Merrick Furst, James B. Saxe, and Michael Sipser. “Parity, circuits, and the polynomial-time hierarchy”. In: *Proceedings of the 22nd Annual Symposium on Foundations of Computer Science*. FOCS ’81. USA: IEEE Computer Society, 1981, pp. 260–270.
- [9] Michael Hahn, Andreas Krebs, Klaus-Jörn Lange, and Michael Ludwig. “Visibly Counter Languages and the Structure of NC^1 ”. In: *Mathematical Foundations of Computer Science 2015*. Ed. by Giuseppe F. Italiano, Giovanni Pighizzini, and Donald T. Sannella. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 384–394.
- [10] Neil Immerman. *Descriptive Complexity*. Springer Verlag, 1998.
- [11] Neil Immerman and Susan Landau. “The Complexity of Iterated Multiplication”. In: *Inf. Comput.* 116.1 (1995), pp. 103–116.

- [12] Andreas Krebs, Klaus-Jörn Lange, and Michael Ludwig. “Visibly Counter Languages and Constant Depth Circuits”. In: *32nd International Symposium on Theoretical Aspects of Computer Science (STACS 2015)*. Ed. by Ernst W. Mayr and Nicolas Ollinger. Vol. 30. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2015, pp. 594–607.
- [13] Clemens Lautemann, Pierre McKenzie, Thomas Schwentick, and Heribert Vollmer. “The Descriptive Complexity Approach to LOGCFL”. In: *Journal of Computer and System Sciences* 62.4 (2001), pp. 629–652.
- [14] David A. Mix Barrington, Neil Immerman, and Howard Straubing. “On uniformity within NC1”. In: *Journal of Computer and System Sciences* 41.3 (1990), pp. 274–306.
- [15] R. Smolensky. “Algebraic methods in the theory of lower bounds for Boolean circuit complexity”. In: *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*. STOC ’87. New York, New York, USA: Association for Computing Machinery, 1987, pp. 77–82.
- [16] H. Straubing, D. Therien, and W. Thomas. “Regular Languages Defined with Generalized Quantifiers”. In: *Information and Computation* 118.2 (1995), pp. 289–301.
- [17] Howard Straubing. *Finite automata, formal logic, and circuit complexity*. CHE: Birkhauser Verlag, 1994.
- [18] H. Venkateswaran. “Properties that characterize LOGCFL”. In: *Journal of Computer and System Sciences* 43.2 (1991), pp. 380–404.